PAPELERÍA MATRIX

1. RESUMEN EJECUTIVO

Papelería Matrix es un sistema de gestión integral para un establecimiento ubicado en Urb. Chapacua, Cartagena de Indias. Combina una página pública de consulta y un backoffice para administrar servicios, papelería, ropa aliada de Pulguero Glam D&A, operadores de cibercafé, ventas, facturación, ingresos y reportes. El proyecto pertenece a la etapa académica de Tecnología en Desarrollo de Sistemas de Información y de Software y también incluye documentación de ingeniería: requisitos, entrevistas, historias de usuario, diagramas, diseño visual, modelo de datos y scripts SQL.

La arquitectura objetivo combina Java como backend, MySQL como persistencia, HTML/CSS/JavaScript como frontend y JDBC para la comunicación. El patrón previsto es MVC con capas: Config, Controllers, Middleware, Models, Dao, Services, Routes y frontend de vistas, estilos y recursos. La implementación actual es parcial: existe conexión JDBC, modelos Java, DAOs y tablas SQL; varias DAOs tienen firmas implementadas solo como esqueleto, los controladores/rutas/servicios están vacíos y el JavaScript principal contiene ejercicios TODO.

2. Identidad, actores y alcance

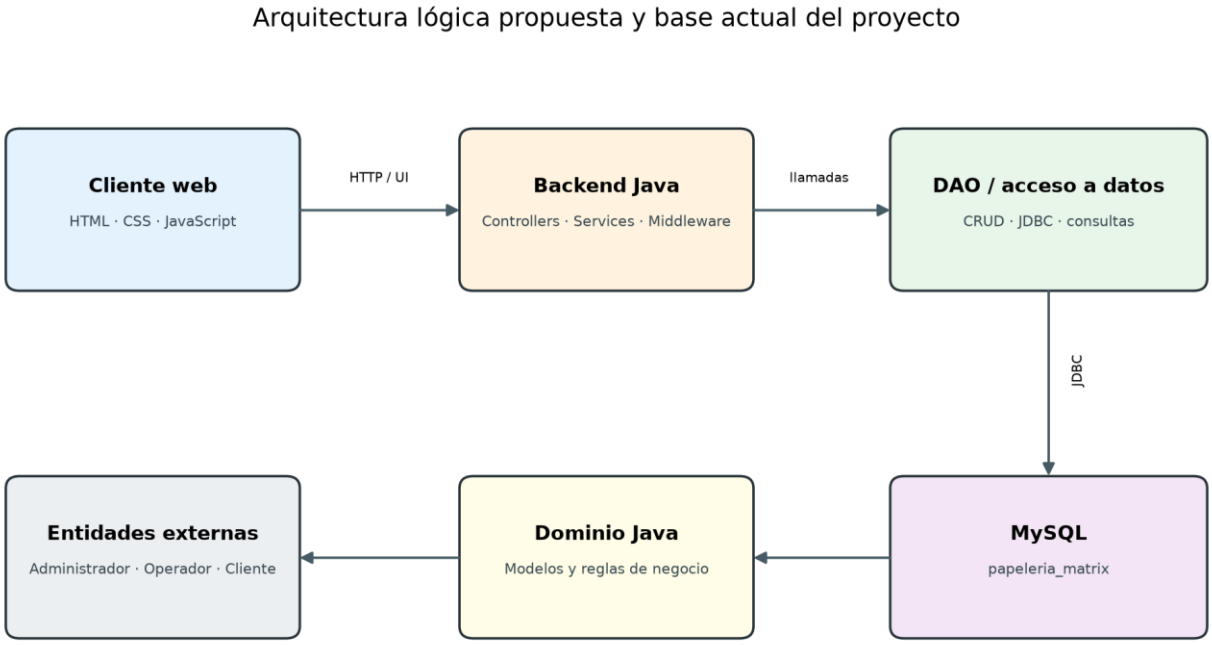
Elemento	*Definición*
Propietaria	Alba Cecilia Botero Zuluaga
Marca aliada	Pulguero Glam D&A
Administrador	Gestiona catálogos, servicios, operadores, información, estadísticas, ingresos y reportes.
Operador de cibercafé	Registra servicios y ventas de papelería/ropa; genera y administra facturas y registros comerciales.
Cliente	Consulta papelería, ropa, servicios, horarios, precios y otra información pública.
Agente AI / actor de apoyo	Revisa mejoras, errores y recomendaciones; no aparece como cliente transaccional ni sustituye roles de autorización.

3. Contexto y objetivos del negocio

- Digitalizar y centralizar procesos manuales de administración, ventas y operación.
- Controlar existencia, entradas y salidas de productos y evitar discrepancias de inventario.
- Mantener catálogos de papelería, servicios y ropa bajo gestión autorizada.
- Registrar facturas y movimientos comerciales, con trazabilidad por fecha y usuario.
- Calcular ingresos y estadísticas, conservar históricos y generar reportes.
- Aplicar autenticación, autorización por rol y seguridad de la información.
- Ofrecer una experiencia pública intuitiva, responsive, accesible y corporativa.

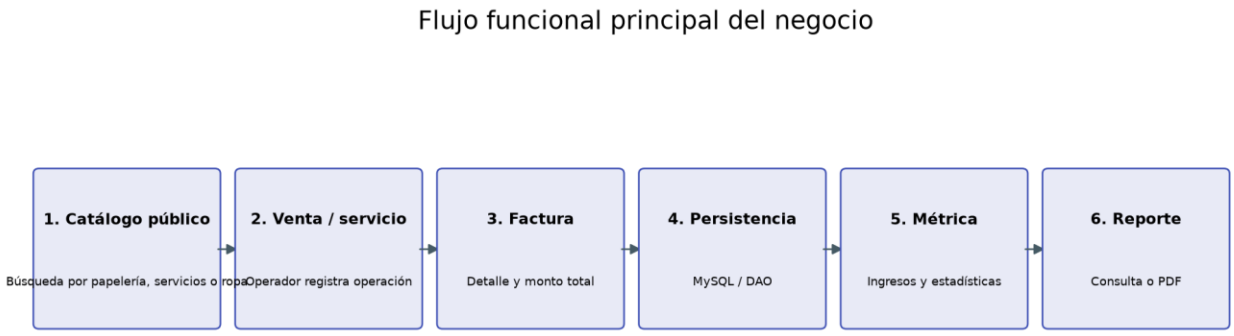
4. Diagramas de sistema

4.1 Arquitectura lógica



La conexión visual representa el diseño previsto. En el código real todavía no existe una cadena completa de vista → ruta → controlador → servicio → DAO → MySQL; esta documentación la identifica como la arquitectura objetivo a completar.

4.2 Flujo principal de negocio



4.3 Modelo de contexto resumido

Administrador ↔ Sistema: solicitudes, cambios maestros, datos, confirmaciones, estadísticas y reportes.
Operador de cibercafé ↔ Sistema: ventas, facturas, registros operativos e historial.
Cliente ↔ Sistema: criterios de búsqueda y consultas; recibe catálogos, resultados y información del negocio.
Agente AI ↔ Proyecto: observaciones, correcciones, recomendaciones y mejoras; debe mantener revisión humana.

5. Tecnologías y decisiones técnicas

Componente	Tecnología / patrón	Estado observado
Backend	Java + JDBC/DAO + POO	Parcial; conexión JDBC y modelos/DAOs presentes
Base de datos	MySQL 8 / XAMPP orientativo	Scripts de creación y FK presentes; importar en orden
Frontend público	HTML5, CSS3, JavaScript vanilla	Maquetación presente; JS mayormente esqueleto TODO
Arquitectura	MVC por capas	Planificada; algunos módulos aún .gitkeep o incompletos
Conector	mysql-connector-j 9.7.0	JAR incluido en Libraries
Diseño	Inter, Space Grotesk, variables CSS, responsive	Recursos y hojas base presentes

Convenciones: lenguaje visible en español; nombres de tablas, columnas, clases, paquetes y variables combinan español y algunos nombres técnicos en inglés. Evitar introducir cambios masivos de idioma o estilo durante la continuidad; mantener compatibilidad y documentar cualquier renombramiento.

6. Lógica de negocio y casos de uso

6.1 Administrador

- Administrar servicios con alta, modificación, eliminación y consulta.
- Administrar productos: código, nombre, cantidad, precio de venta, precio de compra, descripción y procedencia.
- Administrar stock, entradas/salidas y alertas de disponibilidad.
- Administrar información del negocio: horarios, misión, visión, descripción, precios y novedades.
- Contratar, consultar, actualizar y separar operadores; gestionar claves de permiso.
- Administrar catálogo aliado de ropa.
- Consultar ingresos, calcular estadísticas, gestionar históricos y generar reportes, incluidos PDF.

6.2 Operador de cibercafé

- Registrar, consultar, actualizar y eliminar facturas de servicio.
- Generar facturas y registrar ventas de papelería y ropa.
- Ver historial y detalle de operaciones realizadas.

6.3 Cliente público

- Buscar información de papelería, ropa, servicios y del negocio.
- Consultar productos destacados si no hay criterio de búsqueda.
- Abrir sección de login y registro; mostrar horarios y datos de contacto.

6.4 Reglas transversales críticas

- Solo el administrador crea, modifica y elimina servicios, productos maestros, información y operadores.
- El operador debe estar asociado a un Usuario y puede tener una clave de permiso vigente.
- Toda factura debe tener un identificador/código único, fecha y monto; el detalle puede referenciar servicio, producto y/o ropa.
- Las operaciones comerciales deben alimentar ingresos y estadísticas sin perder trazabilidad histórica.
- Las contraseñas internas deben almacenarse cifradas con hash; la tabla Usuario debe guardar hashes, nunca texto plano.
- Una sesión interna se cierra tras 15 minutos de inactividad.

- Los permisos deben evaluarse siempre en servidor; ocultar botones en frontend no sustituye la autorización.
- Los errores técnicos no deben mostrarse al cliente: responder con mensajes claros y registrar detalles internamente.

7. Datos y modelo persistente

La base se llama papeleria_matrix. Contiene 15 entidades principales, organizadas en los siguientes dominios: identidad, venta/facturación, catálogos, operación, gestión de información, ingresos, estadísticas y reportes.

Dominio	Entidades	Relación principal
Identidad	Usuario, Administrador, Operador_de_cibercafe, Clave_de_permiso	Usuario padre especializado para los roles internos
Catálogos	Servicio, Producto_de_papeleria, Ropa	Entidades comerciales independientes consultadas desde detalle y catálogos
Venta	Factura_del_negocio, Detalle_de_factura	La factura contiene cabecera y uno o varios detalles
Información	Informacion_del_negocio	Administrada por Administrador
Ingresos	Ingreso_del_negocio, Historial_ingreso_del_negocio	Historial relacionado con ingreso y motivos
Estadísticas	Estadistica_del_negocio, Historial_estadistica_del_negocio	Historial relacionado con la medición
Reportes	Reporte_del_negocio	Relaciona administrador con históricos de ingresos y estadísticas

Entidad Usuario: claves, nombres, documento, rol, email, teléfono y última sesión. Define los roles Administrador y Operador de cibercafé. Administrador y Operador conservan especializaciones 1:1 mediante Id_usuario; Operador además posee claves de permiso.

Entidad Detalle_de_factura: admite referencias opcionales a Servicio, Producto_de_papeleria y Ropa, cantidades y descripción; permite representar ventas mixtas de servicios, insumos y prendas en una misma factura.

Entidades principales para implementar con cuidado: Factura_del_negocio, Detalle_de_factura, Producto_de_papeleria, Servicio, Ropa, Ingreso_del_negocio, Estadistica_del_negocio e Informacion_del_negocio.

8. Estado real de la implementación

Área	Observado	Trabajo pendiente
Frontend	index.html extensible; CSS modular; diálogo login/registro/búsqueda; navegación responsive en HTML.	Completar JS, validaciones, estados vacíos, resultados y integración API/Java.
JavaScript	Main.js es educativo/esqueleto: selectores y TODO; solo año dinámico resuelto.	Implementar menú, modales, filtros y eventos con accesibilidad.
Backend Java	Conexión, 15 modelos y DAO por entidad.	Usar PreparedStatement, CRUD real, transacciones, manejo de

		recursos, services/controllers/routes.
SQL	15 CREATE TABLE y scripts FK separados.	Corrige diferencias de nombres y orden; crea seeds, índices, integridad y backup.
Pruebas	Carpeta 05-pruebas casi sin automatización visible.	Agregar pruebas unitarias e integración de DAO, reglas y flujos críticos.
Seguridad	Roles y claves incluidos en diseño.	Hash de contraseñas, parametrización SQL, sesión, control de acceso, validación y sanitización.

9. Requisitos funcionales consolidados

- RF catálogo: búsqueda pública de papelería, servicios, ropa y datos del negocio; mostrar información destacada sin criterio.
- RF servicios/productos/ropa: CRUD exclusivo del administrador y consultas para público/autorizados.
- RF información del negocio: agregar, actualizar, eliminar y consultar.
- RF operadores: contratar, consultar, actualizar, despedir, listar y gestionar claves de permiso.
- RF ventas: operador registra y consulta ventas de papelería y ropa.
- RF facturas: operador genera, lista, consulta, actualiza y elimina facturas de servicio.
- RF estadística e ingresos: consultar, calcular, conservar históricos, eliminar con auditoría y generar reportes.
- RF PDF: exportar reportes y facturas digitales cuando corresponda.
- RF usuario: gestionar registro, autenticación, roles y datos personales.

10. Requisitos no funcionales prioritarios

- Seguridad: contraseñas con hash, permisos de servidor, sesiones de máximo 15 minutos y datos válidos.
- Usabilidad: interfaz intuitiva, sin saturación, lenguaje corporativo e inclusión web/accesibilidad.
- Rendimiento: respuestas de búsqueda dentro de 2,5 segundos como objetivo, con código eficiente.
- Responsive: adaptación a distintos tamaños aunque el diseño prioritario no sea móvil.
- Calidad: buenas prácticas, separación por capas, trazabilidad, manejo de errores amigable y GitHub.
- Escalabilidad: estructura preparada para integración futura de IA sin acoplarla al núcleo.
- Datos: integridad referencial y soporte para altos volúmenes sin redundancia innecesaria.
- Disponibilidad: recuperación ante caída de conexión en 15 minutos o menos como objetivo operativo.

11. Diseño visual y experiencia

La marca presenta una interfaz profesional en azul/blanco y acentos corporativos. Debe incluir banner, navegación, secciones Nosotros, Papelería, Pulguero Glam D&A, horarios, catálogos, búsqueda, login y registro. Las imágenes y tipografías están almacenadas bajo Assets. Aplicar componentes BEM y variables CSS; mantener contraste, foco de teclado, etiquetas accesibles y textos alternativos.

12. Instrucciones para continuar el proyecto

- No declarar terminado lo que solamente está maquetado o es un TODO. Distinguir “especificación”, “implementación parcial” y “pendiente”.

- Preservar primero compatibilidad funcional y nombres acordados; corregir errores de integración sin renombrar entidades masivamente.
- Completar verticalmente por caso de uso: SQL → modelo → DAO → servicio → controlador → ruta → interfaz → prueba.
- Usar siempre consultas parametrizadas. No interpolar valores del usuario dentro de sentencias SQL.
- Cerrar Connection, Statement y ResultSet con try-with-resources y devolver objetos o listas, no solo mensajes por consola.
- No guardar contraseñas ni datos sensibles en código, logs o respuestas. Mover credenciales a configuración segura.
- Mantener sesiones, autorización por rol y validaciones servidor-side.
- Agregar pruebas primero para reglas críticas, especialmente facturación, stock, permisos y roles.
- Resolver discrepancias entre documentación y código mediante trazabilidad, conservando el documento como decisión autorizada.

13. Riesgos, inconsistencias y puntos de atención

- Hay archivos SQL y código Java con nombres o casos distintos (Factura_de_negocio vs Factura_del_negocio; Id_usuario vs Usuario_id), que pueden romper compilación o FK. Deben unificarse antes de ejecutar migraciones.
- La clave primaria de Factura usa Usuario_id pero la FK advertida puede apuntar a Operador_de_cibercafe; confirmar la relación de negocio definitiva.
- Algunos DAO solo muestran mensajes de éxito sin ejecutar JDBC ni mapear resultados.
- Las credenciales MySQL están en una clase Java; deben externalizarse.
- El frontend usa una estructura pública, pero aún no hay integración HTTP ni API Java.
- La base contempla reportes en PDF, pero la evidencia de librería y flujo de generación es limitada.
- Existen nombres “Rpa” y posibles errores de escritura de script que requieren revisión de esquema antes de ampliar funcionalidad.

15. Glosario breve

DAO: objeto responsable del acceso a datos.
MVC: separación entre modelo, vista y controlador.
FK: clave foránea que relaciona tablas.
POO: programación orientada a objetos.
CRUD: crear, consultar, actualizar y eliminar.
Seed: datos iniciales de prueba.
Stock: cantidad disponible de un producto.
Catálogo: conjunto consultable de servicios, productos o prendas.

Fuentes internas revisadas

README; requisitos funcionales; requisitos no funcionales; estructura del proyecto; descripción tecnológica; entrevistas y documentación académica; diagramas de contexto, procesos, casos de uso y clases; scripts DDL/FK; modelos Java; DAO; frontend HTML/CSS/JavaScript. La evidencia indica un proyecto en desarrollo y no un sistema plenamente integrado.